HANDS-ON LAB · LECTURE 12 · THE PRACTICAL FOR LECTURE 11

Docker Internals, Proved Live

dockerd, containerd, shim and runc on a live EC2 instance, and the proof, via `/proc`, that a container is just a process.

Sockets · dockerd · containerd · shim · runc · /proc

Interview Q&A



EVERY ARROW · PROVED ON A REAL MACHINE

THE CASE FILE

Why This Matters

> *Yesterday was theory. Today we go looking for evidence: every arrow in the chain, on a real machine.*

Lecture 11 was theory: **Docker CLI → Docker daemon → containerd → runc → process**. Today we **proved every arrow** of that chain on a real machine:

Sockets
the socket files they use to talk

Binaries
the binaries and flags systemd starts

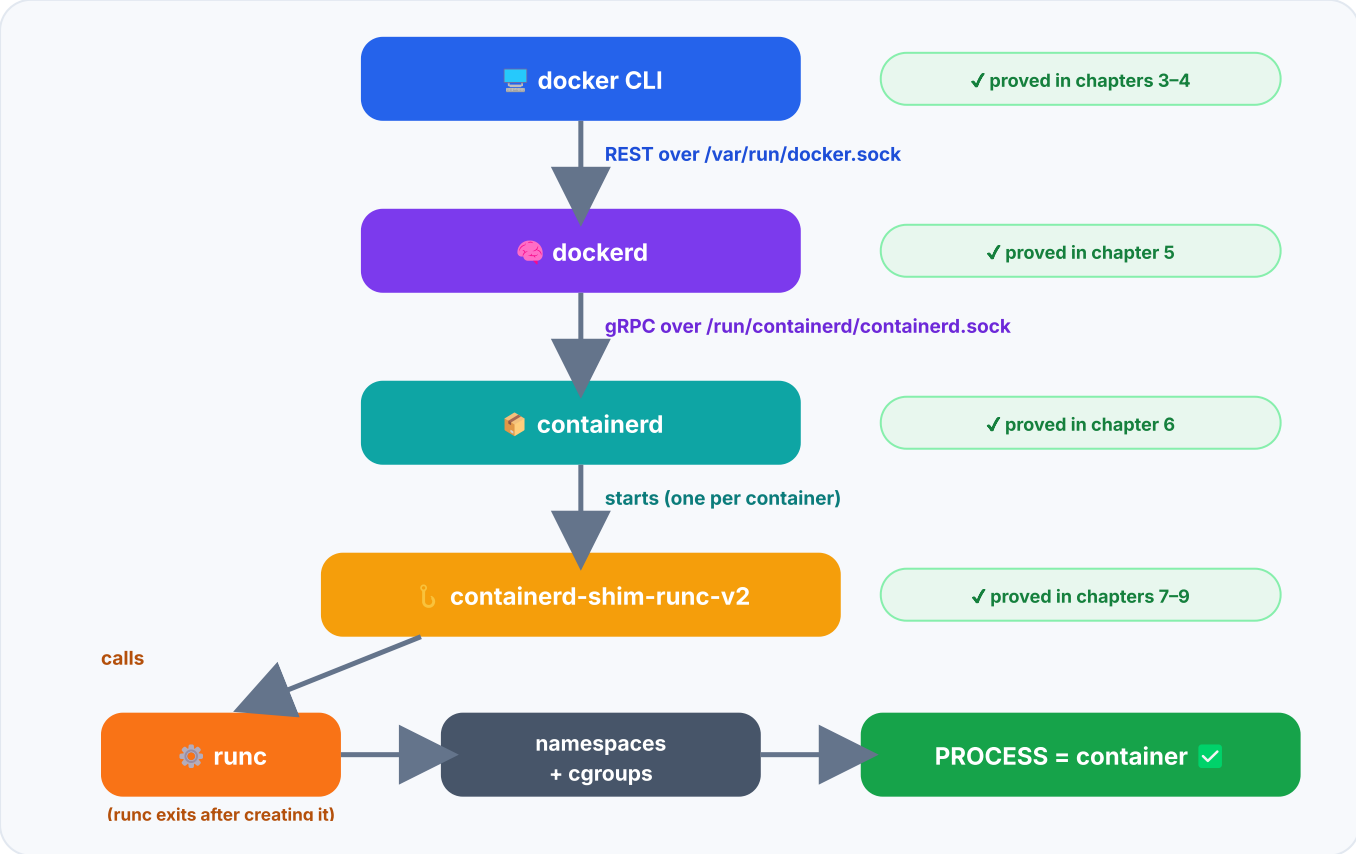
The shim
the shim process that appears when a container runs

/proc
a container is just a Linux process you can find under /proc and kill like any other

This is the evidence behind the interview answer *"what happens when you run* `docker run nginx` *?"*

MACHINE	OS	REGION	ACCESS
EC2 instance	Amazon Linux 2023	ap-southeast-1	SSM Session Manager, as root

Recap: the chain we're going to prove



PART ONE

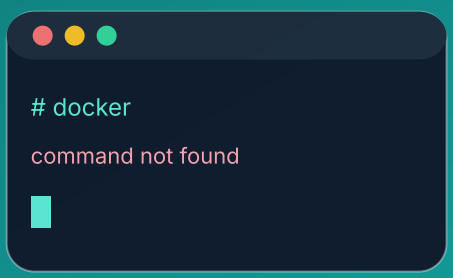
A Fresh Box

An empty EC2 instance. No Docker, nothing listening except SSH. A clean crime scene.

IN THIS PART

✦ 1 · Fresh box: no Docker at all

✦ 2 · `yum install docker` : Docker is not a single thing



01

PART I · STARTING POINT

Fresh Box: No Docker at All

```
root@ip-172-31-7-96
[root@ip-172-31-7-96 ~]# docker
bash: docker: command not found
```

Nothing is installed yet. A clean starting point.

02

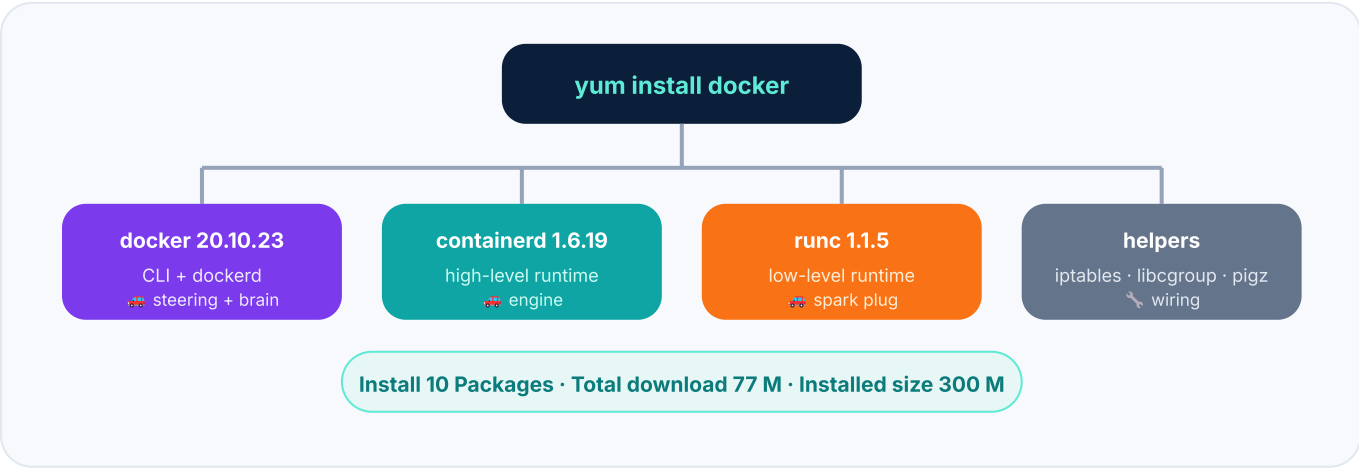
PART I · INSTALLATION

yum install docker: Docker Is Not a Single Thing

> *We asked for one package. Yum delivered a whole car in parts.*

```
install
# yum install docker
```

While installing Docker from the yum repository, we can see it's installing a lot of other things along with it. This is exactly what we discussed yesterday: **Docker is not a single entity**. Like a car, it's many parts, and installing "Docker" installs all of them:



Package	Version (AL2023)	What it is
docker	20.10.23	Docker CLI + Docker daemon (`dockerd`)
containerd	1.6.19	High-level container runtime
runc	1.1.5	Low-level container runtime
iptables-libs, iptables-nft, libnetfilter_conntrack, libnfnetlink, libnftnl	—	Linux firewall/NAT pieces that Docker networking (bridge, port mapping) uses

libcgroup	3.0	Library for cgroups, which runc uses to limit CPU/memory
pigz	2.5	Parallel gzip, to decompress image layers faster

Key point

One `yum install docker` = **CLI + dockerd + containerd + runc + networking and cgroup helpers**. The four players from Lecture 11 are now all on disk.

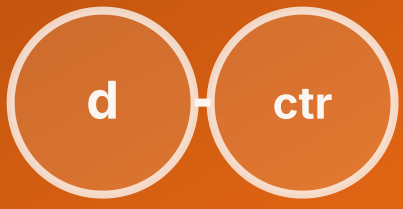
PART TWO

Waking the Daemons

The parts are on disk, but nothing is running. Let's watch them come to life one by one, and read what they say.

IN THIS PART

- ✦ 3 · CLI installed but daemon not running → the socket error
- ✦ 4 · Start Docker: CLI ↔ daemon now works
- ✦ 5 · `systemctl status docker`: reading the dockerd command line
- ✦ 6 · `systemctl status containerd`: the runtime is running too



PART II · EVIDENCE #1

03 CLI Installed, Daemon Not Running → the Socket Error

> *The CLI knocks on the door. Nobody answers.*

```
root@ip-172-31-7-96
[root@ip-172-31-7-96 ~]# docker ps
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon
running?
```



This proves what we learned yesterday:

- The **Docker CLI is only a client**. It tries to talk to the Docker daemon, and the daemon is the background process that should be running all the time to accept instructions from the CLI.
- The CLI can talk to the daemon **only when the daemon is running**.
- `unix:///var/run/docker.sock` is a **Unix socket file used for IPC** (inter-process communication). It's the communication bridge between the Docker CLI and the Docker daemon. The CLI writes its request (a REST API call) into this socket, and the daemon listens on it.

Check what is listening on the box at this point

```
ss -tnlp
[root@ip-172-31-7-96 ~]# ss -tnlp
State   Local Address:Port   Process
LISTEN  0.0.0.0:22           users:(("sshd",pid=2042,fd=5))
LISTEN  [::]:22              users:(("sshd",pid=2042,fd=7))
```

Observation

Only `sshd`. No Docker / containerd process is running yet.

04

PART II · EVIDENCE #2

Start Docker: CLI ↔ Daemon Now Works

> *One systemctl command, and two daemons wake up.*

start

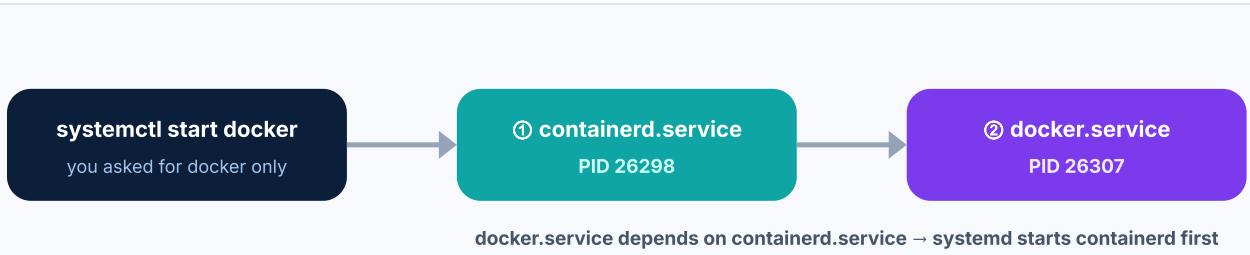
```
# systemctl start docker
# docker ps
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

No error now, just an empty table (no containers yet). **The CLI reached the daemon through the socket.** ✓ ARROW 1 PROVED

`ps aux` now shows **two new long-running processes**:

ps aux

```
root  26298 ... /usr/bin/containerd
root  26307 ... /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --
default-ulimit nofile=32768:65536
```



Notice

We only started `docker`, but **containerd came up too**. `docker.service` depends on `containerd.service`, so systemd starts containerd first (containerd PID **26298** < dockerd **26307**).

PART II · EVIDENCE #3

05 systemctl status docker: Reading the dockerd Command Line

> *Every daemon leaves a confession in its own command line. We just have to read it.*

```
systemctl status docker
```

```
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: disabled)
     Active: active (running) since Sat 2023-06-24 16:24:33 UTC
TriggeredBy: ● docker.socket
    Process: 26305 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
    Process: 26306 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited,
status=0/SUCCESS)
   Main PID: 26307 (dockerd)
      Tasks: 7 (limit: 1114)
     Memory: 31.4M
        CPU: 286ms
     CGroup: /system.slice/docker.service
             └─26307 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
--default-ulimit nofile=32768:65536
```

It gives a lot of information: the **process number**, how many **tasks** are running under Docker, **memory and CPU** usage, and the **CGroup line**. The CGroup line shows the **exact binary + flags** systemd used when we ran `systemctl start docker`. At the back end, this executable is what actually starts Docker:

```
/usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
--default-ulimit nofile=32768:65536
```

the daemon binary · where it listens for the CLI · the next hop: containerd · open-file limit

Part	Meaning
`/usr/bin/dockerd`	The Docker daemon binary
`-H fd://`	`-H` = "host", i.e. where dockerd listens for the CLI. `fd://` = file descriptor: **don't open a network port**. Instead use the socket that systemd already created and handed over (that's why the status says `TriggeredBy: docker.socket`). So dockerd listens to the Docker CLI over that socket, not over the network.

`--containerd=/run/containerd/containerd.sock`	**This is the next hop.** The daemon talks to containerd through another socket file, `/run/containerd/containerd.sock` (gRPC). This is the **"file between daemon and containerd"** we were looking for. ✓ ARROW 2 PROVED
`--default-ulimit nofile=32768:65536`	Default open-file limit (soft:hard) for every container

Useful lines from the daemon log at the bottom of the status

NETWORK

docker0

`Default bridge (docker0) is assigned with an IP address 172.17.0.1/16` : Docker's default network

STORAGE

overlay2

`graphdriver(s)=overlay2` : image layers are stacked with overlay2 (the btrfs error just before it is harmless; Docker tries btrfs first and falls back)

SOCKET

API listen

`API listen on /run/docker.sock` : the CLI socket (`/var/run` is a symlink to `/run`)

⚠ **disabled; preset: disabled**

Docker is started but **not enabled**, so after a reboot it won't come back. In real servers use `systemctl enable --now docker` .

PART II · EVIDENCE #4

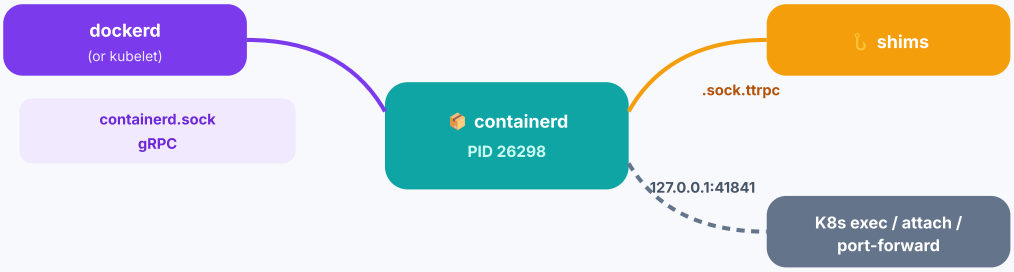
06 systemctl status containerd: the Runtime Is Running Too

> *The engine is idling quietly, waiting for its first job.*

```
systemctl status containerd
```

```
● containerd.service - containerd container runtime
     Loaded: loaded (/usr/lib/systemd/system/containerd.service; disabled; preset: disabled)
     Active: active (running) since Sat 2023-06-24 16:24:32 UTC; 6min ago
    Process: 26295 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
   Main PID: 26298 (containerd)
     CGroup: /system.slice/containerd.service
             └─26298 /usr/bin/containerd
... msg=serving... address=/run/containerd/containerd.sock.ttrpc
... msg=serving... address=/run/containerd/containerd.sock
... msg="Start cni network conf syncer for default"
... msg="Start streaming server"
```

- Before starting, it loads the **overlay kernel module** (needed for the overlay2 layered filesystem).
- It serves on **two sockets**: `containerd.sock` (gRPC, used by dockerd / kubelet) and `containerd.sock.ttrpc` (a lightweight protocol used with the shims).



So now **two processes** are already running: `dockerd` and `containerd`. `ss -tnlp` now shows containerd as well:

```
ss -tnlp
```

```
LISTEN  127.0.0.1:41841   users:(("containerd",pid=26298,fd=13))
```

At the back end, containerd is in **listening mode**.

What is that TCP port?

That `127.0.0.1:41841` TCP port is containerd's **CRI streaming server** ("Start streaming server" in the log). It's used for **exec / attach / port-forward** when Kubernetes talks to containerd. It's bound to localhost only on a random port. The main dockerd → containerd talk goes over the **Unix socket**, not this port.

PART THREE

A Container Is Born

We type docker run. A shim appears, runc flashes in and out, and a brand-new Linux process takes its first breath.

IN THIS PART

- ✦ 7 · Run a container: now runc and the shim come into the picture
- ✦ Resources come from the host
- ✦ 8 · Container = process: kill it and the process is gone



PART III · EVIDENCE #5

07 Run a Container: runc and the Shim Appear

> *Still no container running (docker ps is empty). Now...*

docker run

```
[root@ip-172-31-7-96 ~]# docker run -itd nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
5b5fe70539cd: Pull complete      ← 7 layers pulled (this is containerd's job)
...
4a85ce26214d: Pull complete
Digest: sha256:593dac25b7733ffb7afe1a72649a43e574778bf025ad60514ef40f6b5d606247
Status: Downloaded newer image for nginx:latest
ef22ea64a308293355bf5ccf1aa5463f3400c2124f84779da2488280b2768df8   ← container ID
```

`-i`	interactive
`-t`	TTY
`-d`	detached / run in background

Till now we have seen the Docker CLI, the Docker daemon and containerd. **As soon as the container starts, runc comes into the picture.** Our container got a unique ID (the long SHA above). At the back end runc creates one process, and that process is known as the container.



The new process in ps aux

```
root 27017 ... Sl 16:34 /usr/bin/containerd-shim-runc-v2 -namespace moby
-id ef22ea64a308293355bf5ccf1aa5463f3400c2124f84779da2488280b2768df8
-address /run/containerd/containerd.sock
```

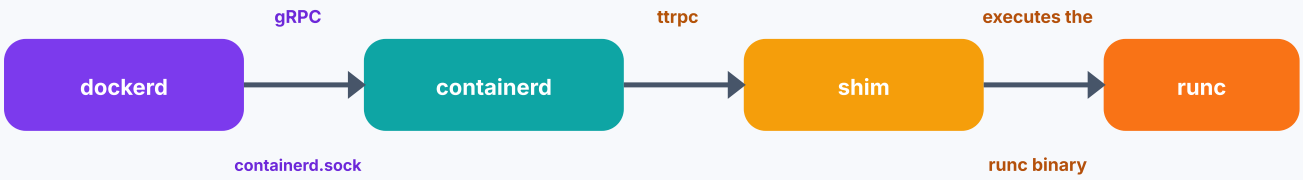
Part	Meaning
`containerd-shim-runc-v2`	**The shim**: a small helper process containerd starts per container. It calls runc to create the container.
`-namespace moby`	A **containerd namespace**, a label containerd uses to keep different clients' containers apart. Docker's containers live in `moby` (Moby = Docker's open-source project). Kubernetes uses `k8s.io`, and `ctr` uses `default`. **This is not a Linux namespace.**

`-id ef22ea64a308…`	The **same container ID** that `docker run` printed. Every container gets a unique ID, and containerd tracks it by this ID.
`-address /run/containerd/containerd.sock`	The containerd socket the shim reports back to (container events, exit status).

Correction on who the process is

The line we found is **the shim, not runc itself**. runc is a **short-lived tool**: the shim runs `runc create/start`, runc sets up namespaces + cgroups, starts the container process, and then **exits**. That's why you never see a runc process sitting in `ps`. The **shim stays** as the container's parent (holds STDIO, reports the exit code), which is why restarting dockerd/containerd doesn't kill running containers.

The full chain of communication

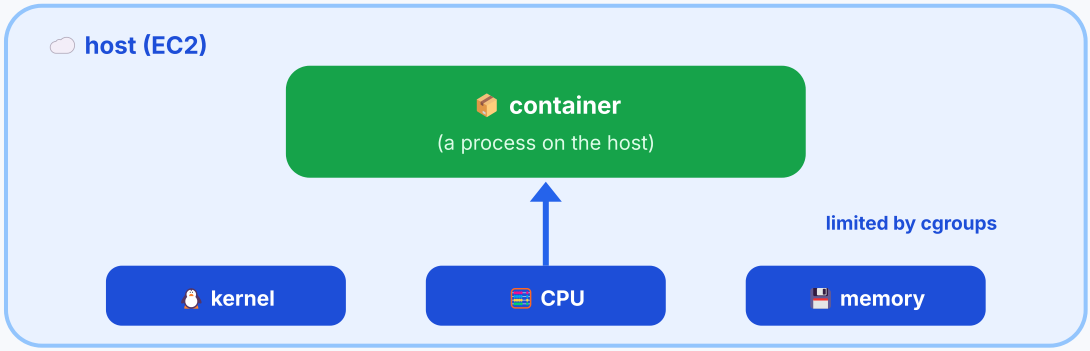


dockerd → containerd over containerd.sock (gRPC) → containerd → shim over ttrpc → shim → runc by executing the runc binary.

✓ ARROWS 3 + 4 PROVED

Resources come from the host

Once the container is created, whatever CPU, memory, etc. it needs to run, it takes from the **host OS**. Our container takes its resources from the EC2 instance it's running on. There is **no separate kernel or "mini VM"**: the container shares the host kernel, and **cgroups** decide how much CPU/memory it may use (unlimited by default unless you pass `--memory` / `--cpus`).



PART III · EVIDENCE #6

08 Container = Process: Kill It and the Process Is Gone

> *The ultimate test: if a container is just a process, killing one must kill the other.*

docker ps

```
[root@ip-172-31-7-96 ~]# docker ps
CONTAINER ID   IMAGE   COMMAND                  CREATED          STATUS          PORTS    NAMES
ef22ea64a308   nginx   "/docker-entrypoint.…"   11 minutes ago   Up 11 minutes   80/tcp   adoring_ishizaka
```

(`docker ps` shows the **first 12 characters** of the same ID. The name `adoring_ishizaka` is **auto-generated** because we didn't pass `--name` .)

docker kill

```
[root@ip-172-31-7-96 ~]# docker kill ef22
ef22
```

(A short unique prefix of the ID is enough.)

Running `ps aux` again: the shim process **27017** for that container is **no longer there**. It has been killed along with the container.



No process means no container. 137 = 128 + 9

So we can say: container = process

Once we kill the container, its process is terminated. **No process means no container.**

(`docker kill` sends **SIGKILL**. `docker ps -a` would show it as `Exited (137)` , where 137 = 128 + 9. `docker stop` sends **SIGTERM** first, waits 10 s, then SIGKILL.)

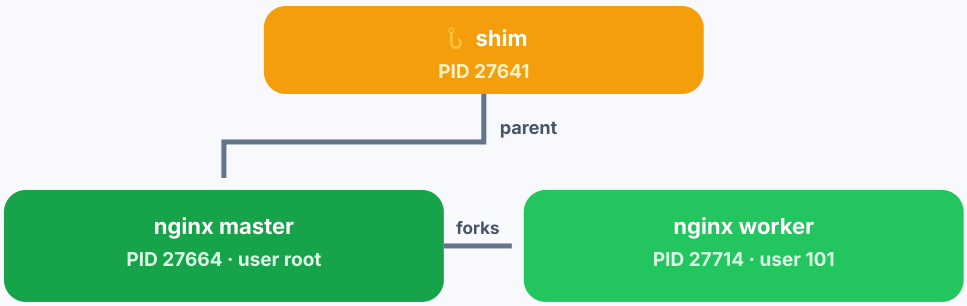
Run again → a brand-new process

docker run again

```
# docker run -itd nginx
80cf84c436793baded9309a74815d5e1e7a35641c187839bb9414c787f858a1a
```

No pull this time; **the image is already cached**. `ps aux` now shows a new process with a new container ID:

```
ps aux
root  27641 ... Sl   /usr/bin/containerd-shim-runc-v2 -namespace moby -id
80cf84c436793baded9309a74815d5e1e7a35641...
root  27664 ... Ss+  nginx: master process nginx -g daemon off;
101   27714 ... S+   nginx: worker process
```



The full family, visible from the host like any other Linux process.

Here we can see the full family: the **shim (27641)**, and under it the actual container processes, **nginx master (27664)** and **nginx worker (27714)**, visible from the host like any other Linux process.

✓ CONTAINER = PROCESS

🎯 Two interview-worthy details in those 3 lines

- The **nginx master runs as** `root`. Without user namespaces, root inside the container is root on the host. That's why we run containers as **non-root** and **drop capabilities**.
- The worker shows **user** `101`: inside the image that UID is the `nginx` user, but the host has no user with that ID, so `ps` prints the number. **Same kernel, same UID table.**

PART FOUR

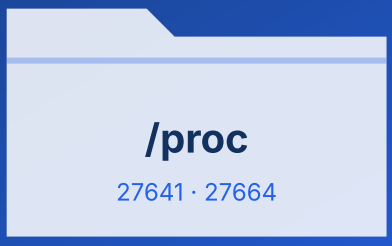
IV

Inside /proc

Every process leaves a folder in /proc. We open the drawer, plant a file called technohulk, and catch an important mistake.

IN THIS PART

- ✦ 9 · Looking inside /proc
- ✦ The technohulk experiment
- ✦ What it really proves (important correction)
- ✦ Comparing namespaces directly



PART IV · EVIDENCE #7

09 Looking Inside /proc

> *In Linux, every running process has a directory under /proc with all its details, named by its PID.*

/proc

```
# cd /proc
# ls      # numbered dirs = PIDs: ... 26298 (containerd) 26307 (dockerd) 27641 (shim) 27664
27714 (nginx) ...
# cd 27641
# ls
arch_status  cgroup  cmdline  cwd  environ  exe  fd  limits  maps  mounts  ns  oom_score
root  status  task ...
```

Inside the process directory we get much more information:

`cmdline`
the command

`exe`
the binary

`ns`
the namespaces it belongs to

`cgroup`
its resource group

`fd`
open files

`root`
the root filesystem as this process sees it

...and more
environ, limits, maps, mounts, status, task

one folder
per PID, for every process

The technohulk experiment 🤦

session 1 · through /proc

```
# cd /proc/27641/root
# ls
bin boot dev etc home lib lib64 local media
mnt opt proc root run sbin srv sys tmp usr
var
# cd tmp/
# touch technohulk
# pwd
/proc/27641/root/tmp
```

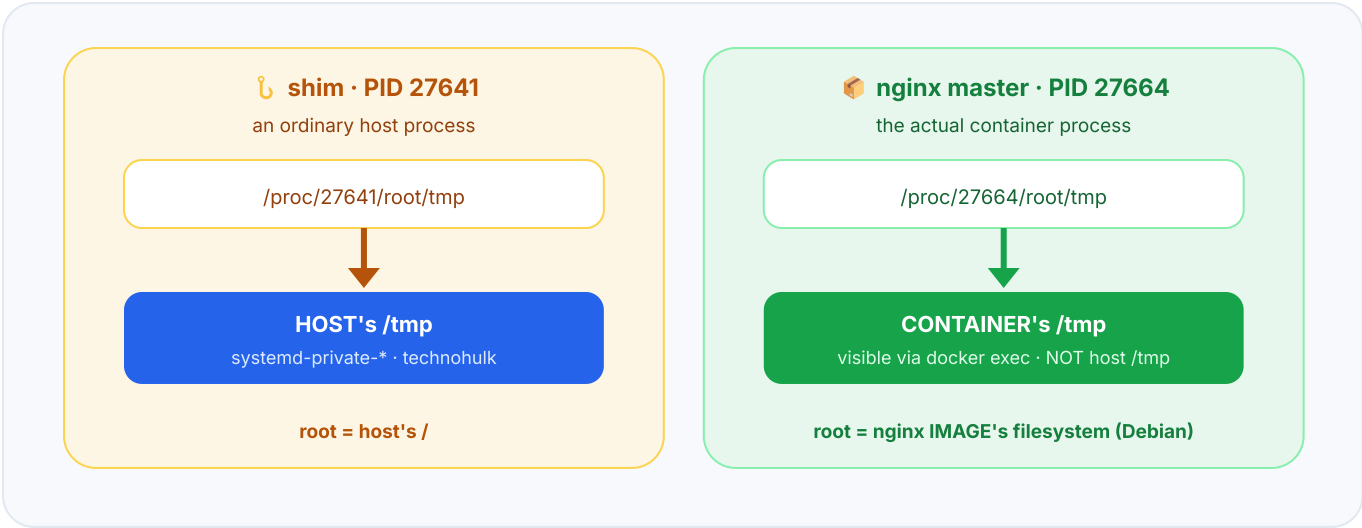
session 2 · as root, not via /proc

```
# cd /tmp
# ls
systemd-private-...-chronyd.service-FbqrZN
...
technohulk   ← the file is here
```

The file created through `/proc/27641/root/tmp` shows up in the **host's** `/tmp`.

⚠️ **What this really proves (important correction)**

PID 27641 is the shim, not the container. The shim is an ordinary host process, so its root is the host's `/`. That's why its `/tmp` had the host's `systemd-private-*` folders and why `technohulk` appeared in the host's `/tmp`. It does **not** mean the container uses the host's filesystem.



Repeat it with the actual container process, nginx master 27664

```
the real container

# ls /proc/27664/root/                # the nginx IMAGE's filesystem (Debian), not the host's
# ls /proc/27664/root/etc/nginx       # nginx.conf etc. (doesn't exist on the host)
# touch /proc/27664/root/tmp/inside   # appears in `docker exec <id> ls /tmp`, NOT in host
/tmp
```

The container has **its own root filesystem** (the image layers stacked by overlay2, isolated by the mount namespace). What it shares with the host is the **kernel**, plus **CPU/memory controlled by cgroups**.

Compare namespaces directly

Different inode numbers = different namespaces:

```
namespaces

# ls -l /proc/27641/ns     # shim  → same mnt/pid/net/uts as PID 1 (host)
# ls -l /proc/27664/ns     # nginx → its own mnt, pid, net, uts, ipc
# ls -l /proc/1/ns         # host init, for comparison
# docker inspect -f '{{.State.Pid}}' <container-id>   # gives you the real container PID
(27664)
```

PID 1 · HOST INIT

Host namespaces

the reference point

PID 27641 · SHIM

= same as host

same mnt/pid/net/uts as PID 1

PID 27664 · NGINX

≠ its own

its own mnt, pid, net, uts, ipc

Debugging trick

From the host you can reach into a container's filesystem through `/proc/<container-pid>/root` (a handy debugging trick when the image has no shell). **The container cannot see the host's files.**

✓ ISOLATION PROVED

10

PART V · THE VERDICT

Everything in One Table

Case closed. Here's every suspect, where they live, how they talk, and how we caught them.

Component	Process / binary	Talks to next hop via	Seen with
🖥️ **Docker CLI**	`docker` (runs only when you type it)	`/var/run/docker.sock` (REST)	`docker ps` error when daemon is down
🧠 **Docker daemon**	`/usr/bin/dockerd -H fd://`	`/run/containerd/containerd.sock` (gRPC)	`systemctl status docker`
📦 **containerd**	`/usr/bin/containerd`	ttrpc to shims; serves `containerd.sock` + `.ttrpc`	`systemctl status containerd`, `ss -tnlp`
🪝 **shim**	`containerd-shim-runc-v2 -namespace moby -id <ID>` (one per container)	executes runc	`ps aux | grep shim`
⚙️ **runc**	`/usr/bin/runc` (exits after creating the container)	Linux kernel: namespaces + cgroups	installed by yum, never stays in `ps`
✅ **Container**	e.g. `nginx: master process`	—	`ps aux`, `/proc/<pid>`, `docker inspect -f '{{.State.Pid}}'`

PART V · LAB SHEET

11 Command Reference From This Lab

install + start

```
yum install docker -y
systemctl start docker        # containerd is pulled in automatically
systemctl enable --now docker # also survive reboots
systemctl status docker       # CGroup line shows dockerd flags; look for "API listen on
/run/docker.sock"
systemctl status containerd   # "serving... address=/run/containerd/containerd.sock"
journalctl -u docker -f       # daemon logs
```

sockets and listeners

```
ls -l /var/run/docker.sock /run/containerd/containerd.sock
ss -tnlp                      # TCP listeners (containerd's 127.0.0.1 streaming port)
ss -xlp | grep -E 'docker|containerd'   # Unix-socket listeners
```

processes

```
ps aux | grep -E 'dockerd|containerd|shim|nginx'
ps -ef --forest   # see shim → nginx master → worker as a tree
pstree -p         # same idea
```

containers

```
docker run -itd nginx
docker ps / docker ps -a
docker kill <id-prefix>            # SIGKILL → Exited (137)
docker inspect -f '{{.State.Pid}}' <id>
```

containerd's own view of Docker's containers

```
ctr namespaces ls                  # moby
ctr -n moby containers ls
ctr -n moby tasks ls               # task = running process, with its PID
```

/proc

```
ls /proc/<pid>/                    # cmdline, exe, ns, cgroup, root, fd ...
cat /proc/<pid>/cmdline | tr '\0' ' '
ls -l /proc/<pid>/ns
ls /proc/<container-pid>/root      # container's filesystem, seen from the host
```

PART V · EXAM READY

12 Interview Quick Answers

1. "Prove a container is just a process."

Run `docker run -d nginx`, then `ps aux` on the host shows `nginx: master process` with a normal PID, owned by a `containerd-shim-runc-v2` parent. `docker kill` it and the process disappears. It has a `/proc/<pid>` entry like any process.

2. "How does the Docker CLI talk to the daemon?"

Over the Unix socket `/var/run/docker.sock` (REST API). That's why "Cannot connect to the Docker daemon at unix:///var/run/docker.sock" means dockerd isn't running (or you lack permission on the socket).

3. "How does dockerd talk to containerd?"

gRPC over `/run/containerd/containerd.sock`, passed as `--containerd=` in the dockerd command line.

4. "What is containerd-shim and why does it exist?"

One per container. It invokes runc, then stays as the parent after runc exits (STDIO, exit code), so containerd/dockerd can restart without killing containers.

5. "What is -namespace moby?"

A containerd namespace (tenant label). Docker uses `moby`, Kubernetes uses `k8s.io`. Not a Linux namespace.

6. "Why is mounting docker.sock into a container dangerous?"

Whoever can write to that socket controls dockerd, which runs as root, so they effectively have **root on the host**.

7. "Does a container have its own OS?"

It has its own filesystem/userland (from the image) but **shares the host kernel**. CPU/memory come from the host, limited by cgroups.

The chain, now fully proved ✓

CLI → dockerd	REST over **/var/run/docker.sock**: socket error when dockerd was down, an empty table once it was up.
dockerd → containerd	gRPC over **/run/containerd/containerd.sock**: seen in the **--containerd=** flag.
containerd → shim	One **containerd-shim-runc-v2** per container (ttrpc), with **-namespace moby -id <ID>**.
shim → runc	Executes the runc binary. runc sets up **namespaces + cgroups**, then exits.
= process	**nginx master/worker** visible in ps and /proc. Kill it and the container is gone.

Still to come

- Same chain on a Kubernetes node: kubelet → CRI → containerd, using `crictl` and `ctr -n k8s.io`
- cgroups hands-on: `docker run --memory 100m` and watch it under `/sys/fs/cgroup`
- `nsenter -t <pid> -n ip a` : enter a container's namespace from the host

Theory in Lecture 11 · Proof in Lecture 12 · Every arrow verified